

Question 1 (10 points)

An affine transformation on a vector b is an operation with the following form:

$$d = c + Ab$$

where A is a $n \times n$ matrix and b, c, d are vectors of n elements.

This kind of transformation can also be applied to matrix and vectors of bits by substituting multiplications with a bitwise AND and additions with a bitwise XOR, i.e.:

$$d_i = c_i \text{ XOR } \sum_{k=0}^{n-1} (a_{ik} \text{ AND } b_k)$$

Note: the sum of the $(a_{ik} \text{ AND } b_k)$ products is performed using XORs as well.

Write a routine in ARM assembly called `bitwiseAffineTransformation` that receives in input:

- the address of the matrix A
- the 8-bit value of b
- the 8-bit value of c

The routine computes the following affine transformation:

$$\begin{bmatrix} a_{77} & a_{76} & a_{75} & a_{74} & a_{73} & a_{72} & a_{71} & a_{70} \\ a_{67} & a_{66} & a_{65} & a_{64} & a_{63} & a_{62} & a_{61} & a_{60} \\ a_{57} & a_{56} & a_{55} & a_{54} & a_{53} & a_{52} & a_{51} & a_{50} \\ a_{47} & a_{46} & a_{45} & a_{44} & a_{43} & a_{42} & a_{41} & a_{40} \\ a_{37} & a_{36} & a_{35} & a_{34} & a_{33} & a_{32} & a_{31} & a_{30} \\ a_{27} & a_{26} & a_{25} & a_{24} & a_{23} & a_{22} & a_{21} & a_{20} \\ a_{17} & a_{16} & a_{15} & a_{14} & a_{13} & a_{12} & a_{11} & a_{10} \\ a_{07} & a_{06} & a_{05} & a_{04} & a_{03} & a_{02} & a_{01} & a_{00} \end{bmatrix} \times \begin{bmatrix} b_7 \\ b_6 \\ b_5 \\ b_4 \\ b_3 \\ b_2 \\ b_1 \\ b_0 \end{bmatrix} + \begin{bmatrix} c_7 \\ c_6 \\ c_5 \\ c_4 \\ c_3 \\ c_2 \\ c_1 \\ c_0 \end{bmatrix} = \begin{bmatrix} d_7 \\ d_6 \\ d_5 \\ d_4 \\ d_3 \\ d_2 \\ d_1 \\ d_0 \end{bmatrix}$$

Keep in mind the following information:

- Each row of the matrix is stored in a single byte
- The bits in each row of the matrix are ordered left-to-right from most to least significant bit
- The vectors are ordered top-to-bottom from most to least significant bit

Example

The bit matrix A is stored as a vector of 8 bytes in a variable called `transformationMatrix`:

`transformationMatrix     DCB 0xF8, 0x7C, 0x3E, 0x1F, 0x8F, 0xC7, 0xE3, 0xF1`

With $b = 0xAA$ and $c = 0x63$, the routine computes the following transformation:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} d_7 \\ d_6 \\ d_5 \\ d_4 \\ d_3 \\ d_2 \\ d_1 \\ d_0 \end{bmatrix}$$

Computer Architectures

Exam of 29/01/2025

A2

$$a_7 \text{ AND } b = 11111000 \text{ AND } 10101010 = 10101000$$

$$\sum_{k=0}^{n-1} (a_{7k} \text{ AND } b_k) = 1 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 0 = 1$$

$$d_7 = 0 \text{ XOR } 1 = 1$$

$$a_6 \text{ AND } b = 01111100 \text{ AND } 10101010 = 00101000$$

$$\sum_{k=0}^{n-1} (a_{6k} \text{ AND } b_k) = 0 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 0 = 0$$

$$d_6 = 1 \text{ XOR } 0 = 1$$

$$a_5 \text{ AND } b = 00111110 \text{ AND } 10101010 = 00101010$$

$$\sum_{k=0}^{n-1} (a_{5k} \text{ AND } b_k) = 0 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 = 1$$

$$d_5 = 1 \text{ XOR } 1 = 0$$

$$a_4 \text{ AND } b = 00011111 \text{ AND } 10101010 = 00001010$$

$$\sum_{k=0}^{n-1} (a_{4k} \text{ AND } b_k) = 0 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 = 0$$

$$d_4 = 0 \text{ XOR } 0 = 0$$

$$a_3 \text{ AND } b = 10001111 \text{ AND } 10101010 = 10001010$$

$$\sum_{k=0}^{n-1} (a_{3k} \text{ AND } b_k) = 1 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 = 1$$

$$d_3 = 0 \text{ XOR } 1 = 1$$

$$a_2 \text{ AND } b = 11000111 \text{ AND } 10101010 = 10000010$$

$$\sum_{k=0}^{n-1} (a_{2k} \text{ AND } b_k) = 1 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 = 0$$

$$d_2 = 0 \text{ XOR } 0 = 0$$

$$a_1 \text{ AND } b = 11100011 \text{ AND } 10101010 = 10100010$$

$$\sum_{k=0}^{n-1} (a_{1k} \text{ AND } b_k) = 1 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 = 1$$

$$d_1 = 1 \text{ XOR } 1 = 0$$

$$a_0 \text{ AND } b = 11110001 \text{ AND } 10101010 = 10100000$$

$$\sum_{k=0}^{n-1} (a_{0k} \text{ AND } b_k) = 1 \text{ XOR } 0 \text{ XOR } 1 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 0 \text{ XOR } 0 = 0$$

$$d_0 = 1 \text{ XOR } 0 = 1$$

The result of the transformation is $d = 11001001_2$

The assembly subroutine must comply with the ARM Architecture Procedure Call Standard (AAPCS) standard (in terms of parameter passing, returned value, callee-saved registers).

Computer Architectures

Exam of 29/01/2025

A2

Question 2 (8 points)

Extend the previous exercise as follows.

Initialize TIMER1 to count to 0xFFFF. When the counter reaches 0xFFFF, the timer is reset and no interrupt is generated.

When INT0 is pressed:

1. Read the value of the timer counter of TIMER1.
2. Perform the XOR of the most and least significant byte of the value.
Remember that the timer counter is 32 bits long: you have to take into account only on the least significant 16 bits! If the value you read from the timer counter is 0x000042AA, the most significant byte is 0x42 and the least significant is 0xAA.
3. Show the value you obtained on the LEDs.

When KEY1 is pressed:

1. Call `bitwiseAffineTransformation` passing:
 - a. The constant array [0x8F, 0xC7, 0xE3, 0xF1, 0xF8, 0x7C, 0x3E, 0x1F] as matrix A
 - b. the value that is being showed on the LEDs as B
 - c. the value 0x63 as C
2. Show the transformed value on the LEDs, using TIMER0 to blink the LEDs with a period of 0.5 seconds (meaning that they have to stay on for 0.25 seconds and off for 0.25 seconds)